Pimpri Chinchwad Education Trust's
Pimpri Chinchwad College of Engineering
(An Autonomous Institute Affiliated to SPPU, Pune)

DevOps - Kubernetes & Helm Laboratory Assignment 7

Student Name: Rutuja Jagtap | PRN: 123B1B146 | Batch: C1
Department: Computer Engineering | Academic Year: 2026-2027 | Semester: II

Explore Kubernetes Container Orchestration and Deploy an Application Using Helm

Kubernetes Architecture, Control Plane, Worker Nodes, Helm Package Manager, Chart Scaffolding, Template Rendering, and Release Lifecycle Management

1. Aim

To study Kubernetes architecture and understand the structure and commands of Helm for managing Kubernetes applications.

2. Problem Statement

Managing containerized applications across clustered environments using static YAML manifests leads to configuration drift, hardcoded parameters, and complex manual upgrade routines. The task is to examine the distributed architecture of Kubernetes, identify its control-plane and worker-node components, and use Helm as a package manager to scaffold a reusable Helm chart, parameterize deployment templates with values.yaml, and perform automated release installation, inspection, rolling upgrades, and clean resource teardown.

3. Learning Objectives

Area	Learning Objective
Container Orchestration	Understand automated deployment, scaling, networking, service discovery, self-healing, and load balancing.
Kubernetes Architecture	Study distributed master-worker topologies and control-plane vs worker-node architectural roles.
Core Components	Identify responsibilities of API Server, etcd, Scheduler, Controller Manager, kubelet, kube-proxy, and CRI.
Helm Package Manager	Understand Helm charts, values parameterization, templates, dependencies, and release versions.
Chart Authoring & Scaffolding	Create a structured Helm chart (myapp/) with Chart.yaml, values.yaml, and Kubernetes templates.
Release Operations & Verification	Execute helm version, create, template, install, list, upgrade, status, and uninstall commands with kubectl verification.

4. Container Orchestration and Kubernetes Overview

Container orchestration is the automated management, scheduling, networking, and scaling of containerized applications across a cluster of computing nodes. While container engines like Docker run containers on single host machines, production

distributed systems require high availability, automated bin packing, fault tolerance, horizontal pod autoscaling, rolling zero-downtime updates, and self-healing restarts.

Kubernetes (K8s), originally developed by Google, is the open-source industry standard for container orchestration. It abstracts underlying compute, network, and storage hardware into a unified API. By enforcing declarative state reconciliation—where controllers continuously align current cluster state with desired state—Kubernetes automates service discovery, ingress routing, secret distribution, and persistent volume provisioning.

5. Kubernetes Architecture, Control Plane, and Helm Concepts

A Kubernetes cluster consists of two primary operational domains: the Control Plane (master components governing cluster state and scheduling decisions) and Worker Nodes (computing nodes running the actual application workloads encapsulated in Pods).

5.1 Kubernetes Cluster Topology and Helm Interaction

The Control Plane components run either as system daemons or static pods, exposing a unified REST API via kubeapiserver. Helm interacts directly with kube-apiserver over authenticated TLS to install and upgrade parameterized application charts, which are subsequently scheduled onto worker nodes.

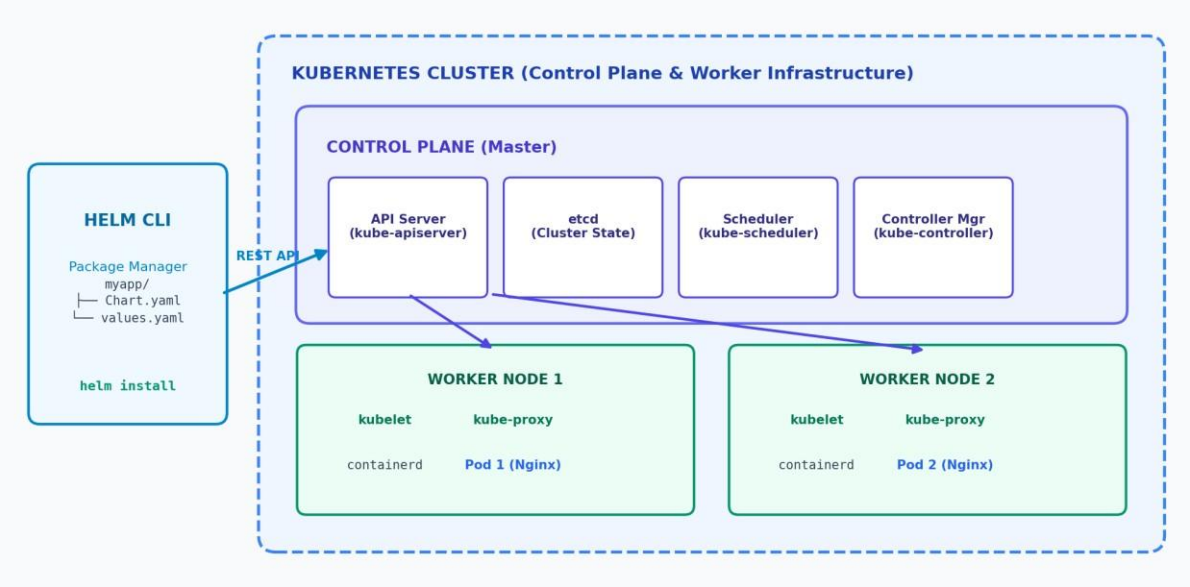


Figure 1. Kubernetes cluster architecture and Helm client package deployment flow

5.2 Kubernetes Component Responsibilities

Component	Architectural Plane	Primary Operational Purpose
API Server (kube-apiserver)	Control Plane	Central entry point exposing the Kubernetes REST API; validates and configures API objects.
etcd	Control Plane	Consistent and highly-available distributed key-value store holding all cluster configuration and live state.
Scheduler (kube-scheduler)	Control Plane	Watches for newly created unscheduled Pods and assigns them to optimal worker nodes based on resources.
Controller Manager	Control Plane	Runs core background daemon controllers (Node, ReplicaSet, Endpoints) to maintain desired cluster state.
kubelet	Worker Node	Primary node agent ensuring containers described in PodSpecs are running, healthy, and reporting status.
kube-proxy	Worker Node	Maintains local network rules and IP tables, enabling Service IP communication and load-balanced pod traffic.

Container Runtime (containerd)	Worker Node	Underlying software responsible for pulling images, allocating namespaces/cgroups, and running containers.
Pod	Workload Object	The smallest deployable computing unit in Kubernetes, hosting one or more closely coupled containers.

5.3 Helm as the Kubernetes Package Manager

Helm acts as the package manager for Kubernetes (analogous to apt for Debian or npm for Node.js). Helm bundles multiple related Kubernetes manifests (Deployments, Services, Ingresses, ConfigMaps) into a single cohesive, versioned package called a Helm Chart. Key concepts include:

- Chart: A bundle of parameterized YAML files describing a Kubernetes application stack.
- Values (values.yaml): Default configuration parameters that can be overridden at install or upgrade time.
- Release: A dedicated running instance of a chart deployed into a Kubernetes cluster with revision history.
- Templates: Dynamic Go-template YAML files rendered with runtime values before submission to the API Server.

5.4 Helm Release Lifecycle Management

Helm manages the entire application lifecycle through discrete phases: chart scaffolding, local templating verification, release installation, parameter updates, and automated uninstallation.

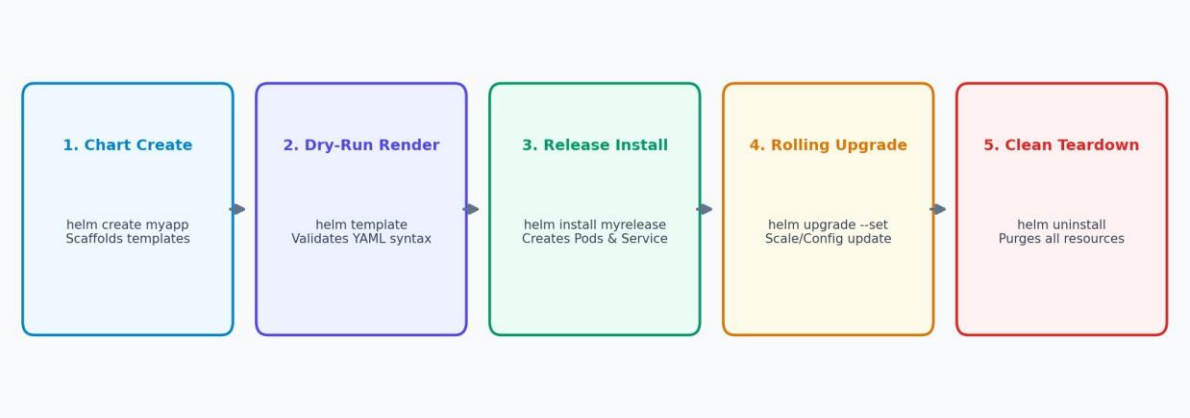


Figure 2. Helm application packaging, deployment, and upgrade lifecycle

6. Installation, Prerequisites, and Environment Configuration

The practical requires an active Kubernetes cluster (such as Minikube or cloud Kubernetes), the kubectl CLI client, and the Helm v3 binary installed.

6.1 Prerequisites Matrix

Prerequisite Component	Minimum Required Version	Verification Purpose
Kubernetes Cluster	v1.28.0+ (Minikube / MicroK8s)	Provides running Control Plane and Worker Node environment.
kubectl CLI	v1.28.0+	Administrative command-line tool for inspecting Kubernetes cluster resources.
Helm v3 CLI	v3.12.0+	Package manager client responsible for chart templating and release management.
Container Runtime	containerd v1.7+	Executes Pod container workloads on node infrastructure.

6.2 CLI Verification Commands

Confirm that Helm is installed and that kubectl is properly authenticated with the cluster:

Cluster & Tool Verification

```
helm version kubectl
cluster-info kubectl get
nodes -o wide
```

6.3 Helm Chart Directory Structure

Executing helm create myapp scaffolds a standardized directory structure:

File / Subdirectory	Role in Helm Package	Description
Chart.yaml	Chart Metadata	Declares chart API version (v2), package name, version, and appVersion.
values.yaml	Default Values	Specifies configurable parameters (replicas, image, service type, ports).
File / Subdirectory	**Role in Helm Package**	**Description**
charts/	Chart Dependencies	Directory for storing dependent sub-charts.
templates/deployment.yaml	Deployment Manifest	Go template defining ReplicaSet, Pod template, and container ports.
templates/service.yaml	Service Manifest	Go template exposing Pods internally or via NodePort/ClusterIP.
templates/_helpers.tpl	Template Helpers	Shared Go template partials and reusable naming macros.

7. Helm Chart Metadata and Configurable Values

The chart package is governed by two core files: Chart.yaml defines the package identification, while values.yaml specifies customizable values injected into templates during deployment.

7.1 Chart Metadata (Chart.yaml) **myapp/Chart.yaml**

```
apiVersion: v2 name:
myapp
description: Sample Kubernetes application deployed via
Helm type: application version: 0.1.0 appVersion: "1.0"
maintainers:
  - name: Niraj Patil
    email: niraj.patil@pccoe.edu
```

7.2 Default Values Configuration (values.yaml) **myapp/values.yaml**

```
replicaCount: 2
 image:   repository:
nginx   pullPolicy:
IfNotPresent   tag:
latest
 service:
type: NodePort
port: 80
targetPort: 80
 resources:
limits:
    cpu: 200m
memory: 256Mi
requests:
cpu: 100m
memory: 128Mi
```

8. Kubernetes Template Manifests Implementation

The templates/ directory contains dynamic Kubernetes YAML manifests. Helm uses the Go text/template engine to evaluate variables from values.yaml, generating valid Kubernetes API schemas.

8.1 Deployment Template (templates/deployment.yaml)

myapp/templates/deployment.yaml

```
apiVersion: apps/v1 kind:
Deployment
```

```
metadata:
  name: {{ .Release.Name }}-{{ .Chart.Name }}
labels:     app.kubernetes.io/instance: {{
.Release.Name }}     app.kubernetes.io/name: {{
.Chart.Name }} spec:   replicas: {{
.Values.replicaCount }}   selector:     matchLabels:
app.kubernetes.io/instance: {{ .Release.Name }}
template:     metadata:       labels:
app.kubernetes.io/instance: {{ .Release.Name }}
spec:       containers:
- name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag
}}"           imagePullPolicy: {{ .Values.image.pullPolicy }}
ports:
- containerPort: {{ .Values.service.targetPort }}
```

8.2 Service Template (templates/service.yaml)

myapp/templates/service.yaml

```
apiVersion: v1 kind:
Service metadata:
  name: {{ .Release.Name }}-{{ .Chart.Name }}
labels:     app.kubernetes.io/instance: {{
.Release.Name }} spec:   type: {{
.Values.service.type }}   ports:
    - port: {{ .Values.service.port }}
      targetPort: {{ .Values.service.targetPort
}}       protocol: TCP   selector:
    app.kubernetes.io/instance: {{ .Release.Name }}
```

9. Helm Templating, Dry-Run, and Value Overrides

Before submitting manifests to the Kubernetes cluster, Helm enables developers to render and inspect the interpolated YAML output locally using the helm template command. This prevents syntax errors and verifies dynamic parameter substitutions.

Dry-Run Rendering & Linting

```
helm template myrelease ./myapp --set replicaCount=2 helm
lint ./myapp
```

Helm also supports runtime value overrides without editing files: the --set flag overrides individual keys (e.g., --set replicaCount=3), and -f custom-values.yaml injects entire environment-specific value files (e.g., production vs staging).

10. Execution Workflow

The comprehensive sequence of Helm and kubectl commands executed during the practical:

Step #	Command Executed	Operational Purpose	Verification Point
1	helm version	Checks Helm CLI build information and API version.	Returns v3.15.4 with GitCommit hash.
2	kubectl cluster-info	Verifies Control Plane API server endpoint connectivity.	Returns running Control Plane and CoreDNS URLs.
3	helm create myapp	Scaffolds standard Helm chart directory structure.	Generates Chart.yaml, values.yaml, and templates/.
4	helm template myrelease ./myapp	Locally renders templates without cluster deployment.	Outputs resolved Kubernetes Deployment and Service YAML.
5	helm install myrelease ./myapp	Deploys chart as release 'myrelease' on Kubernetes.	Release status shows deployed (Revision 1).
6	helm list	Lists all active Helm releases in the default namespace.	Displays release myrelease, chart version, and appVersion.
7	kubectl get pods,svc	Inspects live Pods, Deployments, and Services in cluster.	Confirms 2 Pods Running and NodePort service active.

8	helm upgrade myrelease --set replicaCount=3	Performs rolling upgrade scaling replicas to 3.	Release revision increments to 2; 3rd pod starts.
9	helm status myrelease	Queries status, revision history, and notes of the release.	Displays Revision 2 status and deployment manifest.
10	helm uninstall myrelease	Uninstalls release and removes all associated resources.	Pods and Service terminated cleanly; release purged.

11. Security, Best Practices and Good Practices

Practice	Kubernetes & Helm Implementation & Application
Values Separation	Never store environment-specific or secret parameters in templates; separate into values.yaml or external Vault.
Linting & Dry-Runs	Execute helm lint and helm template before running helm install in production pipelines to catch schema errors.
Resource Quotas	Always declare resources.requests and limits in templates to prevent rogue pods starving worker node resources.
Deterministic Versioning	Adhere to Semantic Versioning (SemVer 2.0.0) in Chart.yaml (version for chart package, appVersion for container).
Namespace Isolation	Deploy applications into dedicated Kubernetes namespaces (-n mynamespace) rather than the default namespace.
Atomic Operations	Utilize the --atomic flag during install and upgrade so failed deployments automatically roll back to previous healthy revisions.

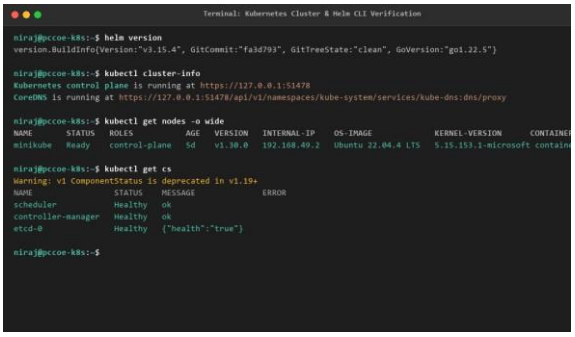
12. Practical Verificatio ı Checklist

Activity	Verification Point	Evidence to Attach
K8s & Helm Setup	Kubernetes cluster healthy; Helm v3 installed and verified.	CLI version and cluster-info screenshot (Evidence A).
Chart Scaffolding	myapp/ chart directory created; helm template renders YAML cleanly.	Chart file tree and template output screenshot (Evidence B).
Release Installation	helm install succeeds; 2 Pods running and Service accessible.	Helm release list and kubectl get pods screenshot (Evidence C).
Upgrade & Teardown	helm upgrade scales replicas to 3; helm uninstall removes all objects.	Upgrade scaling and uninstall confirmation screenshot (Evidence D).

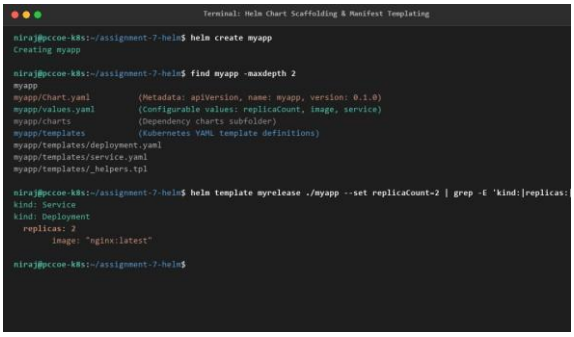
13. Practical Evidence

The terminal captures below record the complete lifecycle of Kubernetes container orchestration and Helm application management, verifying cluster health, chart templating, release deployment, rolling upgrade scaling, and clean teardown:

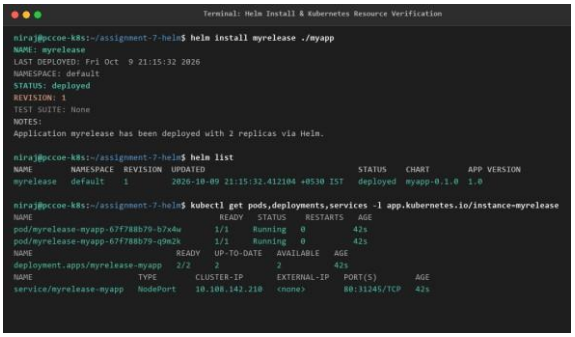
A. Helm & Kubernetes Environment Verification



B. Helm Chart Creation & Dry-Run Templating



C. Helm Chart Installation & Pod / Service Verification



D. Helm Release Upgrade, Status & Clean Uninstall



14. Result

The laboratory assignment successfully demonstrated container orchestration principles and Kubernetes cluster architecture, as well as package management using Helm. The roles of the Control Plane (API Server, etcd, Scheduler, Controller Manager) and Worker Nodes (kubelet, kube-proxy, Container Runtime) were comprehensively evaluated. A custom Helm chart (myapp) was scaffolded, parameterized using values.yaml, rendered locally via helm template, deployed into the cluster as release 'myrelease', scaled from 2 to 3 replicas via helm upgrade, and cleanly uninstalled.

15. Conclusion

Kubernetes provides a robust, self-healing substrate for enterprise container orchestration, while Helm delivers the necessary abstraction for packaging, versioning, and deploying complex microservice topologies as reusable, repeatable code. By parameterizing Kubernetes manifests and managing releases declaratively, Helm eliminates configuration sprawl and enables predictable GitOps workflows across development, staging, and production environments. The practical equips students with essential skills in distributed cluster management, manifest templating, and automated release lifecycle operations.